MATS Summer 2026: Simplex Work Test

Joe Emerson

March 15, 2026

1 Summary

I investigated activation geometry of a small transformer model trained on a non-ergodic training set composed of token sequences generated by two distinct Mess3 processes. I found that the belief geometries of both Mess3 processes were linearly encoded in the model. I also found that the dimensionality of the residual stream roughly matched the predicted dimensionality formed by composing the subspace out of independent subspaces for each process. However, my prediction that the activation geometries for each process would live in orthogonal subspaces was incorrect. I attributed this to the short context window which did not permit a high level of certainty in the identity of the process that generated the sequence. To investigate this further, I created a second model with a longer context window and used two new Mess3 processes that were more distinguishable at smaller contexts. This second model showed higher dimensionality in the residual stream and less robust encoding of the belief states. The subspaces for each process were still highly overlapping and were not orthogonal, although they were not perfectly aligned. With sufficiently large context windows and more distinguishable generative processes, I still expect transformers to encode the belief geometries for each process in orthogonal subspaces. However, given my compute limitations, I was not able to show this empirically. This would be an interesting direction for future study, because it could provide a geometrical description of in-context learning for language models.

2 Prompt

Please implement the following experiment, and send us a PDF writeup of the results, alongside the code. It may be helpful to refer to our publications for definitions of processes and how to derive belief state geometry, in particular, Transformers Learn Factored Representations and the references therein. For computational feasibility, we recommend making the neural networks small (e.g., 1–3 layers, context window 8–16).

1. Construct a non-ergodic training dataset, consisting of Mess3 processes with different parameters, where each training sequence is generated en-

tirely by one Mess3 ergodic component. Train a transformer on this data via next-token prediction. Why do you think this type of structure is interesting and/or relevant to language models?

2. Make a pre-registered prediction (honor code) as to what geometry the activations should take, and how it should change with context position and across layers. Derive this prediction mathematically as far as you can, and separately give your intuition for what the geometry should look like. You will not be penalized for getting this wrong—we are interested in both your formal reasoning and your geometric intuition. Are there multiple possible geometries you can think of?

3. After training, analyze the residual stream geometry. What structure is there? How does it relate to the belief geometries of the component processes?

4. Perform at least one additional analysis of your choosing that you think is interesting or informative. Tell us why you chose it. If you don't have time to implement it, describe what you would do.

3 Why Non-ergodic Training Data is Interesting

Ergodicity is a property of a random system in which the long-term average behavior of a single trajectory is the same as the average of an ensemble of many systems at a fixed point in time. An ergodic system tends to visit all possible states, while non-ergodic systems do not.

Why is this interesting for modeling text? This is important because text corpora tends to be non-ergodic. That is, any one sample will not have the same average statistics as an ensemble of all samples at a particular token. The statistics of a given string of text depends on the process that generated it. This process will be different for different corpora. Therefore, the average over different processes at a fixed point has different statistics than the long-run average for any given process.

This is relevant to language models, because given that real text on the internet is non-ergodic, to optimally predict next tokens, they must not only infer the average statistics, but also infer which process has generated the string. This is called in-context learning, because it must use context to change which statistics to use in its prediction.

Mess3 is a hidden Markov Model (HMM) with three hidden states and three observed states. An individual Mess3 is an ergodic system. The statistics that the sequences take depends on the parameterization of the transition matrices. Mess3 provides a good model for language both because we can control the parameters of the model and because the optimal beliefs about the hidden states given only the observed states can be derived, giving us a ground truth to compare against.

Providing multiple Mess3 processes to generate data for the same training models how different text sources contribute to learning and in-context learning for language models trained on large heterogeneous text databases.

4 Predictions

4.1 Predicting the Activation Geometry

4.1.1 Intuition

The language model must minimally track the probabilities of the next token to make accurate predictions. There are three tokens in the alphabet, so probabilites can be encoded by 2 dimensions (lose one dimension because we stipulate that probabilities must sum to 1).

But the transition matrices will depend on which process generated the data. In order to best predict the next token, the language model must also track which process is likely to have generated the current context.

How might the language model encode probabilities about the next token in the residual stream?

First what would be the dimensionality of the space needed to encode predictions of the next token?

1. Hypothesis 1: Add probability encoding dimensions

   Given that there are 2 dimensions needed to encode probabilities over tokens given a 3-token alphabet, we could imagine that the minimal number of dimensions needed to encode the token probabilities *and* the process that generated them would be to add the process probabilities in as extra dimensions. Given K process and a $(D+1)$-token alphabet, the model could encode these in a $(D+K-1)$-dimensional subspace.

2. Hypothesis 2: Factorized representations

   Given the apparent inductive bias of transformers towards factorized representations (Shai et al., 2026), we also might expect that the model may encode the prediction states in orthogonal subspaces. In this case, we have a DK-dimensional subspace.

3. Hypothesis 3: Joint representation

   Lastly, it is possible that the model represents the data in a joint subspace between each possible process. The dimensionality of this subspace would be $(D+1)K-1$.

4.1.2 Reasoning about hypotheses

To get a geometric intuition, I'll imagine an alphabet size of $D+1=3$ and $K=2$ independent processes generating the data. On hypothesis 1, we have a 3D subspace to encode the beliefs. The belief vectors for each process can

be encoded in a 2D space. The remaining dimension then would encode the probability that process 1 or process 2 generated the current context. We can imagine this as two parallel planes constituting the subspace for each of the processes. As we move between these two planes, we would be interpolating between the predictive vectors in each. The geometries are therefore coupled.

For the second hypothesis, the predictive states for each potential process are encoded on orthogonal planes in a 4D space. Since the same context may admit multiple potential processes, the actual activations would live off of these planes. The projections of these activation vectors would encode the belief state according to each process. But the probability would be encoded jointly by the remaining two dimensions.

For the third hypothesis, each state-process combination is encoded on a unique dimension.

4.1.3 Which hypoethesis is the most plausible?

The problem with the first one is that it does not permit the model to make independent updates conditional on each process. Instead, the current predictive vector is a mixture between processes. In reality, the sequence could only have been generated by a single process. This can affect how the belief trajectory evolves, and will likely result in a averaging between the belief geometries rather than a stable representation of all of the belief geometries.

The second hypothesis avoids the problem of mixed belief states. Instead, the projection of the points onto the manifolds representing each process' belief geometry can accurately track the beliefs of each process conditioned on that process being the one that generated the context. The proximity to each of these orthogonal manifolds then encodes the probability of the process that generated the observed context. The dimensionality scales like DK which is slightly worse than hypothesis 1, but is still linear. This scaling is still better than hypothesis 3, in which the token and process are each tracked on unique dimensions, which is also still linear.

Another piece of evidence for hypothesis 2 is that transformers have already been found to have an inductive bias toward factorized representations when sequences can be described as conditionally independent combinations of factors, and even when conditional independence is relaxed. That in combination with the geometric intuition should bias us toward thinking hypothesis 2 will be correct.

4.2 Predictions across layers and contex

At earlier context positions, the probability of each process given the current context will be more evenly distributed (higher entropy). I'll consider what happens to the activation geometry under hypotheses 1 and 2. Under 1, I would expect the belief geometry to look like an average of the process belief geometries early in the context and then gradually resolve into the appropriate belief geometry as we move through the context. Under hypothesis 2, I would

expect the projections of the activations at each context position to closely resemble the belief trajectory of each process, i.e. the building up of the fractal structure. But, in the residual stream space, I would expect the activations to approach the manifold representing the true process at later context positions. This wouldn't affect the appearance of the linearly-decoded belief geometry though, which would still look very similar to the actual belief geometry.

I think a similar prediction for layers is warranted. Early layers may represent less fully resolved geometries than later layers. Later layers may represent probabilities of the process that generated the sequence.

4.3 Derivations

4.3.1 Set up

An optimal predictor must track the predictive vectors for each sequence. From Shai et al. (2026) we can define the probability of a squence given a GHMM as:

$$Q_{\mathcal{M}}(x_{1:\ell}) = \eta^{\emptyset} T^{(x_1)} ... T^{(x_\ell)} \mathbf{1} \tag{1}$$

where $x_{1:\ell}$ is a sequence of length ℓ, $\eta^{\emptyset}$ is the initial probability row vector over states such that $\eta^{\emptyset}\mathbf{1} = 1$, T^x is the transition matrix for token x, and $\mathbf{1}$ is a column vector of ones to marginalize over states. A predictive vector yielding the probabilities of each state at the current token can be obtained by normalizing by the total sequence probability:

$$\eta^{(x_{1:\ell})} \equiv \frac{\eta^{\emptyset} T^{(x_1)} ... T^{(x_\ell)}}{\eta^{\emptyset} T^{(x_1)} ... T^{(x_\ell)} \mathbf{1}} = \frac{\eta^{\emptyset} T^{(x_1)} ... T^{(x_\ell)}}{Q_{\mathcal{M}}(x_{1:\ell})} \tag{2}$$

The predictive vector provides all the information about the past neccesary to predict the next token. In the current set-up, an optimal predictor will need to track both the predictive vector given each potential process and the probability of the process given the context.

$$\eta^{(x_{1:\ell})} = \sum_{k=1}^{K} P(\mathcal{M}_k | x_{1:\ell}) \eta_k^{(x_{1:\ell})} \tag{3}$$

where $\mathcal{M}_k$ is an individual process of a total of K independent processes that generated the data. $\eta_k^{(x_{1:\ell})}$ is then the predictive vector for each individual process. The probability that the process generating the current context was $\mathcal{M}_k$ is then given by the conditional probability $P(\mathcal{M}_k | x_{1:\ell})$. The conditional probability can be found by marginalizing the sequence likelihood over processes,

$$P(\mathcal{M}_k | x_{1:\ell}) = \frac{P(\mathcal{M}_k) Q_{\mathcal{M}_k}(x_{1:\ell})}{\sum_{k'=1}^{K} P(\mathcal{M}_{k'}) Q_{\mathcal{M}_{k'}}(x_{1:\ell})} \tag{4}$$

where $P(\mathcal{M}_k)$ is the prior probability of $\mathcal{M}_k$. Plugging in the definition of predictive vector and the conditional probability of the process for each process we get:

$$\eta^{(x_{1:\ell})} = \sum_{k=1}^{K} \frac{P(\mathcal{M}_k) Q_{\mathcal{M}_k}(x_{1:\ell})}{\sum_{k'=1}^{K} P(\mathcal{M}_{k'}) Q_{\mathcal{M}_{\|'}}(x_{1:\ell})} \frac{\eta_k^{\emptyset} T_{\mathcal{M}_k}^{(x_1)} ... T_{\mathcal{M}_k}^{(x_\ell)}}{Q_{\mathcal{M}_k}(x_{1:\ell})} = \sum_{k=1}^{K} \frac{P(\mathcal{M}_k) \eta_k^{\emptyset} T_{\mathcal{M}_k}^{(x_1)} ... T_{\mathcal{M}_k}^{(x_\ell)}}{\sum_{k'=1}^{K} P(\mathcal{M}_{k'}) Q_{\mathcal{M}_{k'}}(x_{1:\ell})} \tag{5}$$

The main point is that $\eta^{(x_{1:\ell})}$ can be expressed as a linear combination of $\eta_k^{(x_{1:\ell})}$. By analogy to the Factored World Hypothesis (FWH) from Shai et al. (2026), we can say that since the predictive vectors can be expressed as a linear combination of the possible predictive vectors, each process can exist in an orthogonal subspace.

This supports hypothesis 2, that the activation geometries corresponding to the belief state geometries of the different processes will be represented in orthogonal subspaces. It also shows that for an optimal predictor, the model must track the process probabilities separately from the process-specific predictive vectors.

4.3.2 Across Context

How should I expect the activation geometry to change across context? I gave some vague intuition before. Now I'll see if I can mathematically derive a prediction.

What I can do is try to derive a form of the predictive vector as context length goes to infinity. This should take into a count the steady-state solutions to the processes. As $\ell \to \infty$, we expect the process probability to converge toward the true process.

$$\lim_{\ell \to \infty} P(\mathcal{M}_k | x_{1:\ell}) = \delta(k) \tag{6}$$

$$\delta(k) = \begin{cases} 0 \text{ if } k \neq k^* \\ 1 \text{ if } k = k^* \end{cases} \tag{7}$$

where $\delta(k)$ is a discrete probability distribution that takes on a value of 1 at the true process index k. That means that I can write the predictive vector as $\ell \to \infty$ as well. This will be

$$\lim_{\ell \to \infty} \eta^{(x_{1:\ell})} = \lim_{\ell \to \infty} \sum_{k=1}^{K} P(\mathcal{M}_k | x_{1:\ell}) \eta_k^{(x_{1:\ell})} = \eta_{k^*}^{(x_{1:\ell})} \tag{8}$$

So in the limit, the predictive vector converges to the predictive vector of the true process. This can be read in the geometry as the predictive process converging to the subspace representing the belief state geometry of the true process. We should see over a finite context that subsequent context converge toward this subspace.

5 Results

5.1 Model Details

I used TransformerLens' HookedTransformer to create a small transformer model. Model parameters:

1. Number of layers: 2
2. Number of heads: 2
3. Number of parameters per head: 32
4. Number of MLP parameters: 256
5. Context window: 8
6. Vocabulary size: 3

5.2 Training Details

I optimized model weights using ADAM. The model was trained on next token-prediction with sequences of length 9 (one plus the context window size). Training data were generated by two separate Mess3 processes. On each step, an equal number of sequences were generated by each of the Mess3 processes. The Mess3 processes had parameters ($\alpha = 0.6$,$x = 0.15$) and ($\alpha = 0.6$,$x = 0.2$) respectively. I chose these parameters because they yielded different belief geometries while maintaining a high theoretical maximum accuracy. The theoretical maximum accuracy is the expected probability of predicting the next token given ground truth belief state (i.e. the theoretically optimal belief state given the observed sequence of tokens). For any HMM, this accuracy will be equal to the expected emission probability, i.e. the expected value that the emission probabilities will take under the current beliefs about the hidden states. At any token position, we have three belief probabilities $b = [b_0, b_1, b_2]$. The probability of emitting token x is therefore, $P(x_{\ell+1}) = \sum_{i=0}^{2} b_i P(x_{\ell+1}|x_{1:\ell})$. A perfect predictor will maximize this probability. Therefore, the maximum accuracy is:

$$\mathbb{E}_{b\sim\text{stationary}}[\max_{x_{\ell+1}}(P(x_{\ell+1}|b)] \tag{9}$$

The $b \sim$ stationary here notes the expected value under the stationary belief distribution. This will be different than the stationary distribution for the hidden states (which for Mess3 will be uniform), because the emission probabilities factor into the stationary distribution for the beliefs. That is, under certain emission probabilities, tokens will be more highly indicative of the underlying state than in other regimes, which will make the sequence more predictable. I numerically calculated the maximum accuracies by finding a stationary belief distribution over a long sequence generated by each process, then taking the maximum probability.

The maximum accuracies for each process were 0.440 and 0.400 respectively. Training parameters:

1. Number of steps: 1000
2. Batch size: 25000
3. Learning rate: 5e-4

5.3 Activation Geometry

To analyze the activation geometries, I computed the ground truth belief state for each of a set of test sequences generated from each of the Mess3 processes. Linear projections of the activations in the residual stream can be used to analyze the structure of the activations. Following Shai et al. (2025), a linear map can be formed from the residual space to the hidden states. To get a set of belief states $b \in \mathbb{R}^{|\mathcal{S}|}$, where $|\mathcal{S}|$ is the number of hidden states, from an activation vector $a \in \mathbb{R}^{d_{\text{resid}}}$, where d_{resid} is the number of residual weights, we can use the following:

$$b \approx Wa + c \tag{10}$$

where the matrix $W \in \mathbb{R}^{|\mathcal{S}| x d_{\text{resid}}}$ and bias vector $c \in \mathbb{R}^{|\mathcal{S}|}$ can be found by minimizing the mean squared error between predicted belief states and true belief states. That means for each sequence, we will have a set of activations and a set of ground truth belief states. Each gets a mapping, and the composite set of predicted belief states give me the geometry.

The model recovered the belief geometry in the residual stream of the final layer for both processes.

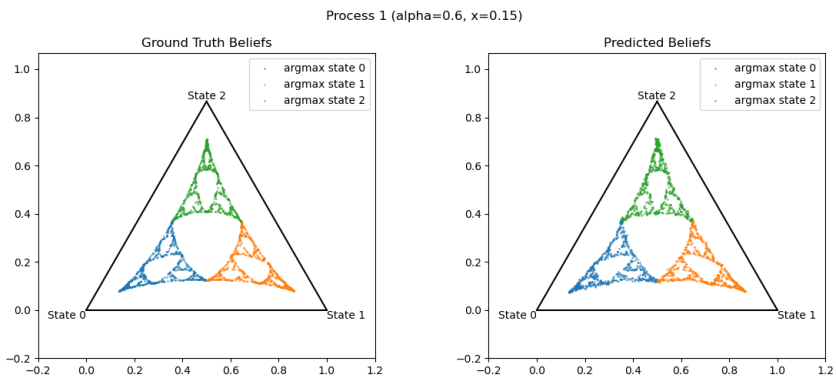

Figure 1: **Linearly Recovered Belief Geometry** ($\alpha = 0.6$**,** $x = 0.15$): Linear projection from residual activations of final layer of transformer model to belief state simplex. Comparison with ground truth belief state geometry on left. Right shows recovered geometry from transformer.

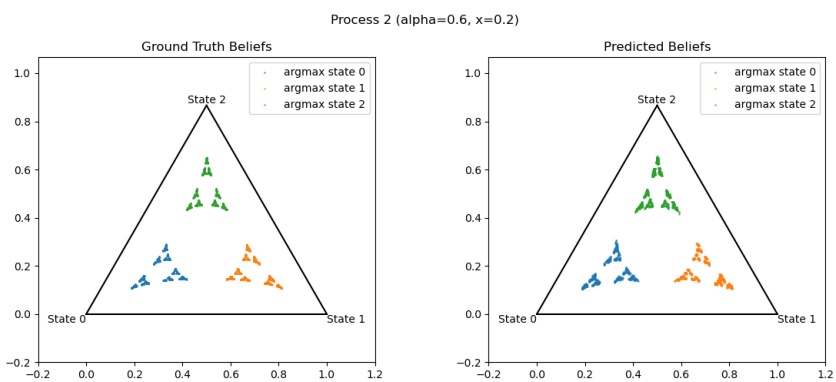

Figure 2: **Linearly Recovered Belief Geometry** ($\alpha = 0.6$**,** $x = 0.2$): Linear projection from residual activations of final layer of transformer model to belief state simplex. Comparison with ground truth belief state geometry on left. Right shows recovered geometry from transformer.

I also analyzed how the geometry changed across context position. The activation geometry evolves nearly identically to the theoretical belief state trajectories in both layers.

In the beginning, you have only one token. Given the known belief transition matrices, or having learned them as in the case of the transformer, we should know to associate this token with a higher likelihood of the corresponding state producing this token. So we end up with three belief probabilities that are possible depending on the first token. On the next step, three tokens are possible. This divides each ofh the three trajectories into three again. The positions they

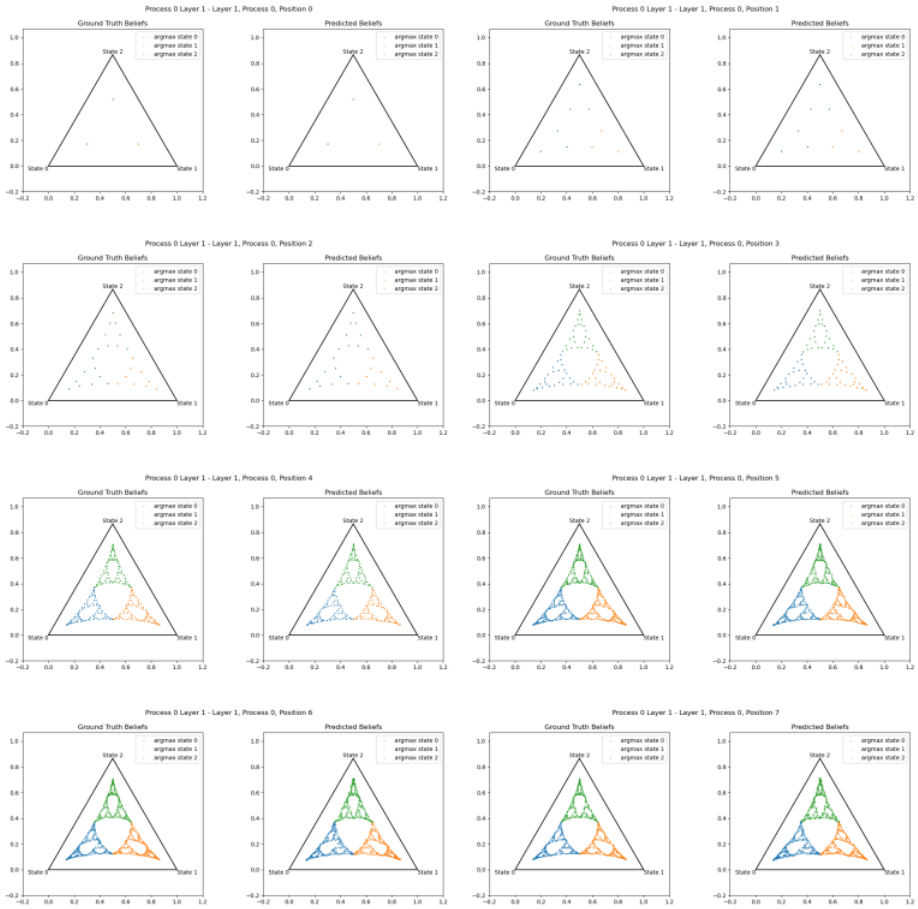

Figure 3: **Activation Geometries Over Context Window**: Linear projection from residual activations of final layer of transformer model to belief state simplex. Each panel shows the projection at each position in the context window.

take will depend on how the transition matrix determines the next step. We can see this spliting continue with more tokens, until we have the full fractal structure.

5.4 Dimensionality

I also analyzed the dimensionality to test my predictions about how beliefs about each process are encoded in the residual stream. I found that the necessary principle components needed to account for 95% of the covariance between activations was around 4 or 5 depending on the layer. This is close to my

predicted dimensionality of 4, but may also be consistent with a joint representation wherein the combinations of states and processes are all represented along separate dimensions.

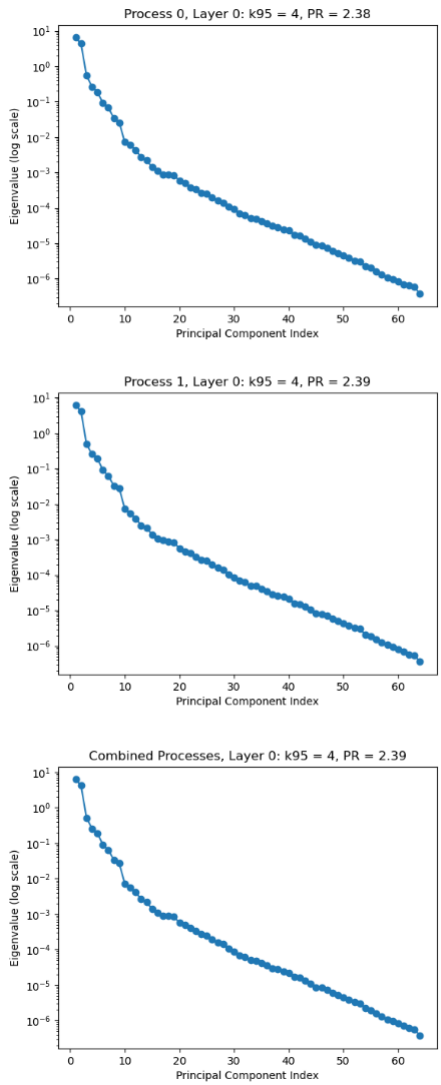

Figure 4: **Eigenspectrum of Residual Stream Covariance**: The eigenspectrum of the residual stream covariance for each process was computed independently and then combined. The k95 indicates the number of components needed to explain 95% of the variance, and participation ratio (PR) is defined as $PR = (\sum_i^N \lambda_i)^2/(\sum_i^N \lambda_i^2)$

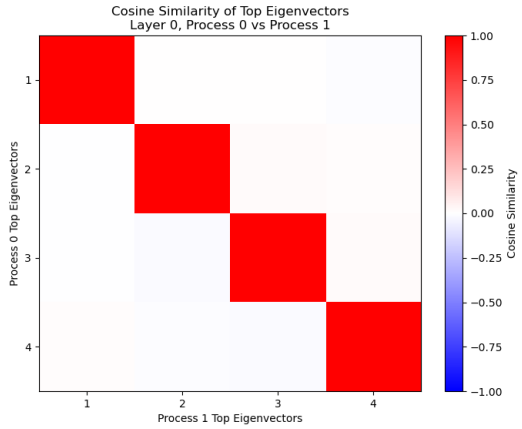

Figure 5: **Cosine Similarity of Process Subspaces (First Layer)**: The subspaces for each process in the residual stream were quantified using the eigenvectors of the covariance matrix. The cosine similarity between the eigenvectors were used to determine the rotation of these subspaces relative to one another.

However, I found that the dimensionality of the activation geometry for each process was also 4 or 5. This indicates that the geometry with full context does not lie on 2D subspaces for each process as I predicted if the assumption that we are close to the infinite context limit. In fact, we have a pretty small context window (8) and at full context, the process probability has not converged to δ_{k^*}.

I also analyzed the eigenvectors of the activation covariance to determine whether the representations of the processes live in orthogonal subspaces. The following plots show the cosine similarity between each pair of eigenvectors of the first and second processes. The diagonal shows strong alignment between the eigenvectors between process activations. This confirms that both geometries share a subspace in the residual stream. Again, my suspicion is that this is due to high uncertainty about the process that generated the sequence given a context of only 8 tokens.

Finally, I also analyzed how dimensionality changes over context position. I found that dimensionality increases across context. Here's what I think is going on. At position 0, we cannot tell the different processes apart at all, so we can encode the belief in two dimensions as if it were a single process that generated the data. That's because the conditional probability of the process will depend only on the prior at this point. After the next token, we now need at least two dimensions to encode the probabilities of the states. On subsequent tokens, extra dimensions are added to encode the relative probabilities of each process responsible for the sequence.

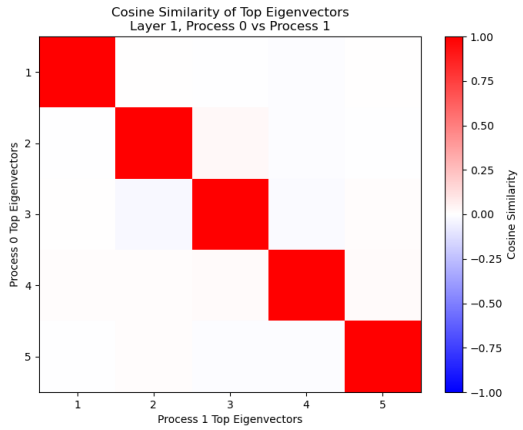

Figure 6: **Cosine Similarity of Process Subspaces (Second Layer)**: The subspaces for each process in the residual stream were quantified using the eigenvectors of the covariance matrix. The cosine similarity between the eigenvectors were used to determine the rotation of these subspaces relative to one another.

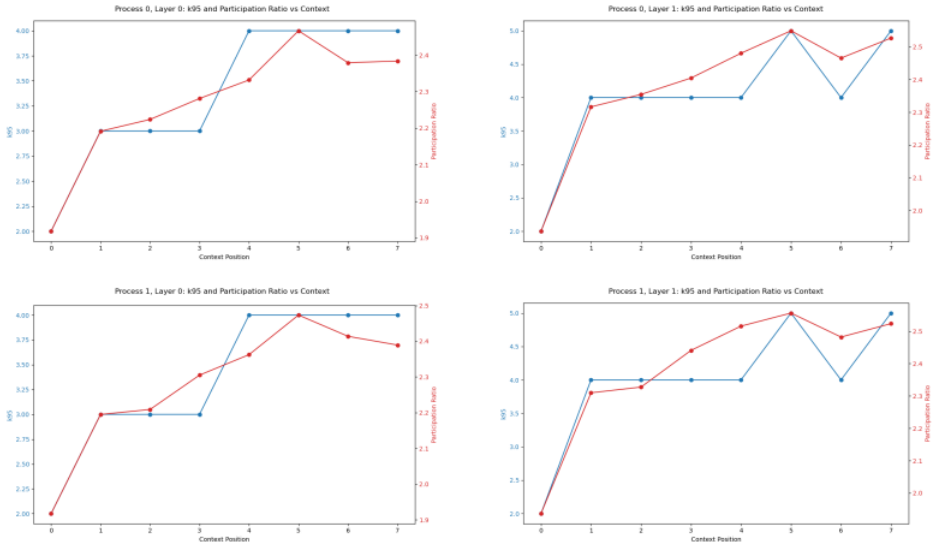

Figure 7: **Dimensionality vs Context Position**: Plotted are k95 and PR as a function of context position.

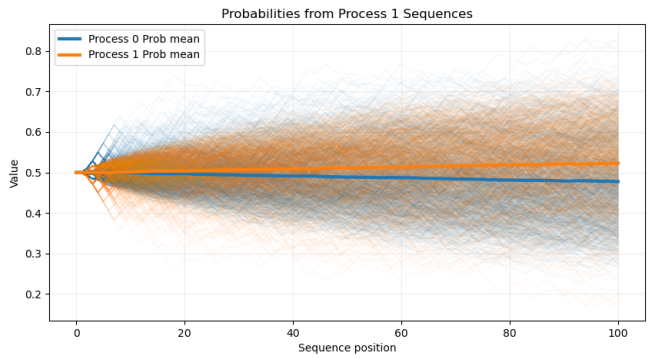

Figure 8: **Process Probability Trajectories**: A thousand sequences were generated from each process and the sequence probability at each token was computed. From the sequence probabilities, the process probabilities were computed $P(\mathcal{M}_{\|}|x_{1:\ell})$. The average probabilities over all sequences at each token are shown in bold with individual processes shown with thin lines. Process 0: ($\alpha = 0.6$, $x = 0.15$), Process 1: ($\alpha = 0.6$,$x = 0.2$)
.

6 Further Analyses

I believe that my prediction about the orthogonality of subspaces representing different processes is due to a failure to converge toward strong evidence for one process over the other given a context window of only 8. To investigate this, I computed the relative probabilities of each process given a sequence. I then computed this as a function of token position to observe how quickly we converge toward certainty about the process given a long series of tokens. The conditional probability of a process given a sequence generated by one of K possible processes $\mathcal{M}_k$ is given by equation 4. I assume a uniform prior probability over processes.

The processes I chose are not distinguishable with high probability with an 8 token context window. Even going out to 100 tokens only brings probabilities modestly above/below 50%. So simply increasing the context window to 16 from 8 will probably not affect the geometry.

I tested two new processes that I thought might yield more distinguishable sequences. These have parameters ($\alpha = 0.6$,$x = 0.15$) and ($\alpha = 0.2$,$x = 0.4$). Indeed, these probabilities diverge more quickly. Still a 16-token context will not yield perfect distinguishability, but perhaps I can glimpse a transition toward an orthogonal representation by using these different processes plus a longer context.

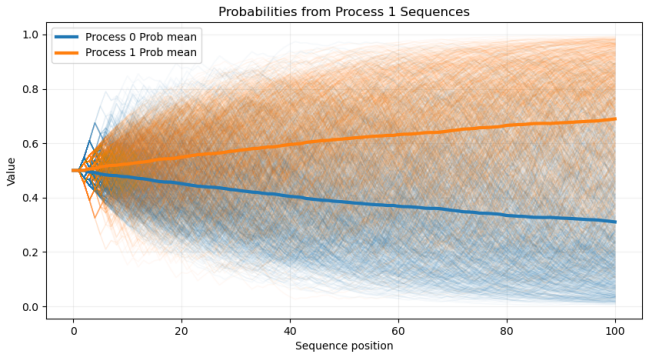

Figure 9: **Process Probability Trajectories (new processes)**: A thousand sequences were generated from each process and the sequence probability at each token was computed. From the sequence probabilities, the process probabilities were computed $P(\mathcal{M}_{\|}|x_{1:\ell})$. The average probabilities over all sequences at each token are shown in bold with individual processes shown with thin lines. Process 0: ($\alpha = 0.6$, $x = 0.15$), Process 1: ($\alpha = 0.2$,$x = 0.45$).

6.1 Second Training Run

The second training run had the same parameters. All of the model parameters were the same except for the context window, which was increased to 16.

6.2 Results

I still find that the geometries of both are learned by the model, although not as perfectly as the first run. This training run may have been more challenging due to a lower theoretical maximum accuracy. There were also more parameters in the model, so the training may have not fully converged.

The dimensionality of the hidden layers was surprisingly large, $k^*_{95} = 6$ or 7.

Looking at the eigenvectors, the subspaces for each process appear to overlap. However they do seem slightly rotated relative to one another within the joint subspace unlikely previously.

Looking at the dimensionality over context, I saw a similar trend as previously.

In summary, I think we are still far from the large context limit, so I don't think I have successfully tested my hypothesis. If I were to continue this, I would try either 1) identifying different processes that quickly converge to the long-context limit, or 2) train a model with a much larger context window on Mess3 data. Identifying whether models converge toward orthogonal activation geometries in the long-context limit could help explain how in-context learning is achieved in large language models. This could help identify how models shift between domains/personas/etc. and could have interpretability implications.

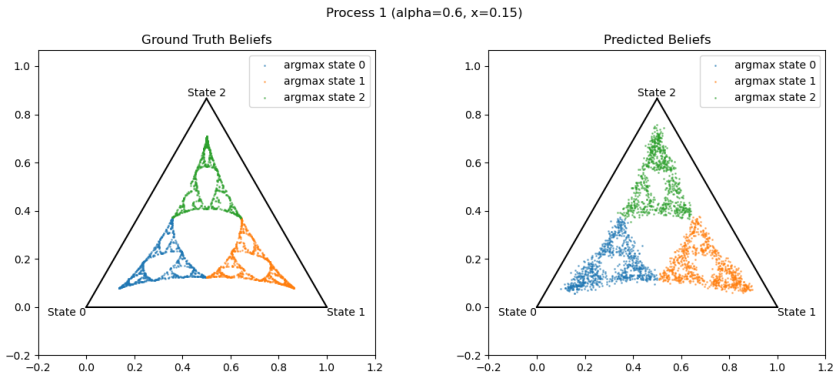

Figure 10: **Linearly Recovered Belief Geometry with Long Context ($\alpha = 0.6$, $x = 0.2$)**: Linear projection from residual activations of final layer of the long context (n_ctx=16) transformer model to belief state simplex. Comparison with ground truth belief state geometry on left. Right shows recovered geometry from transformer.

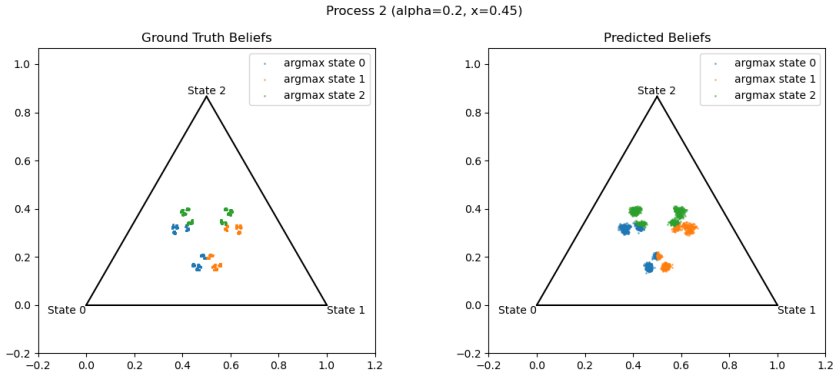

Figure 11: **Linearly Recovered Belief Geometry with Long Context ($\alpha = 0.2$, $x = 0.45$)**: Linear projection from residual activations of final layer of the long context (n_ctx=16) transformer model to belief state simplex. Comparison with ground truth belief state geometry on left. Right shows recovered geometry from transformer.

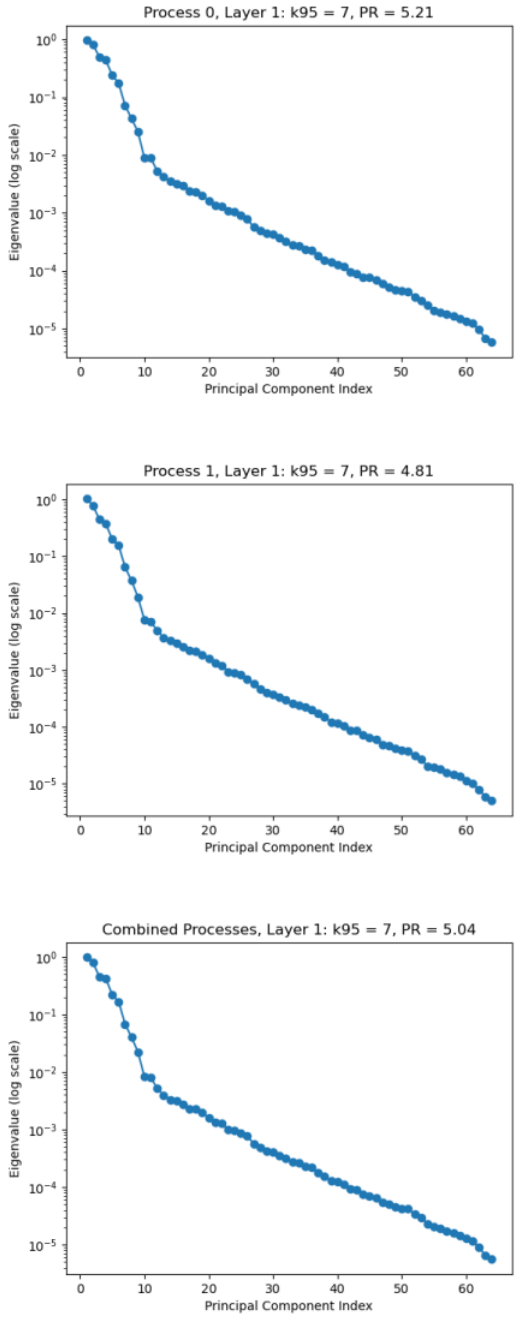

Figure 12: **Eigenspectrum of Residual Stream Covariance for Long Context**: The eigenspectrum of the residual stream covariance for each process was computed independently and then combined. The k95 indicates the number of components needed to explain 95% of the variance, and participation ratio (PR) is defined as $PR = (\sum_i^N \lambda_i)^2/(\sum_i^N \lambda_i^2)$

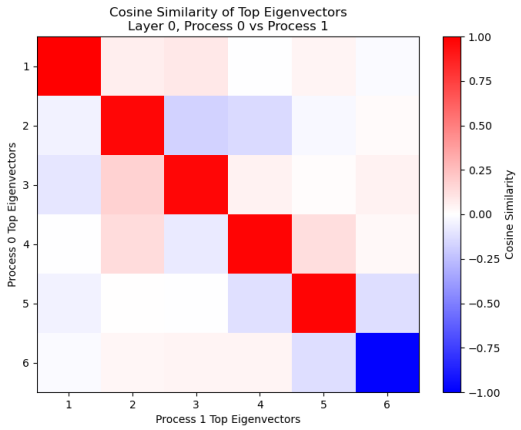

Figure 13: **Cosine Similarity of Process Subspaces for Long Context (First Layer)**: The subspaces for each process in the residual stream were quantified using the eigenvectors of the covariance matrix. The cosine similarity between the eigenvectors were used to determine the rotation of these subspaces relative to one another.

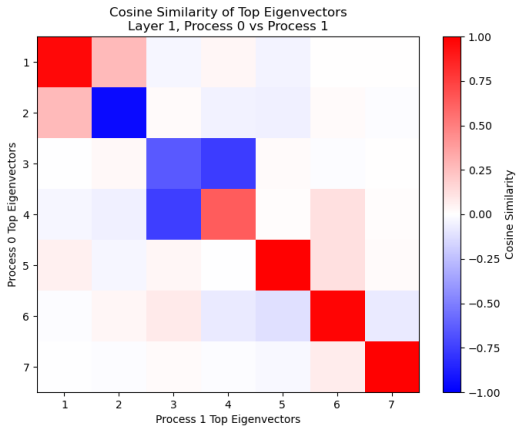

Figure 14: **Cosine Similarity of Process Subspaces for Long Context (Second Layer)**: The subspaces for each process in the residual stream were quantified using the eigenvectors of the covariance matrix. The cosine similarity between the eigenvectors were used to determine the rotation of these subspaces relative to one another.

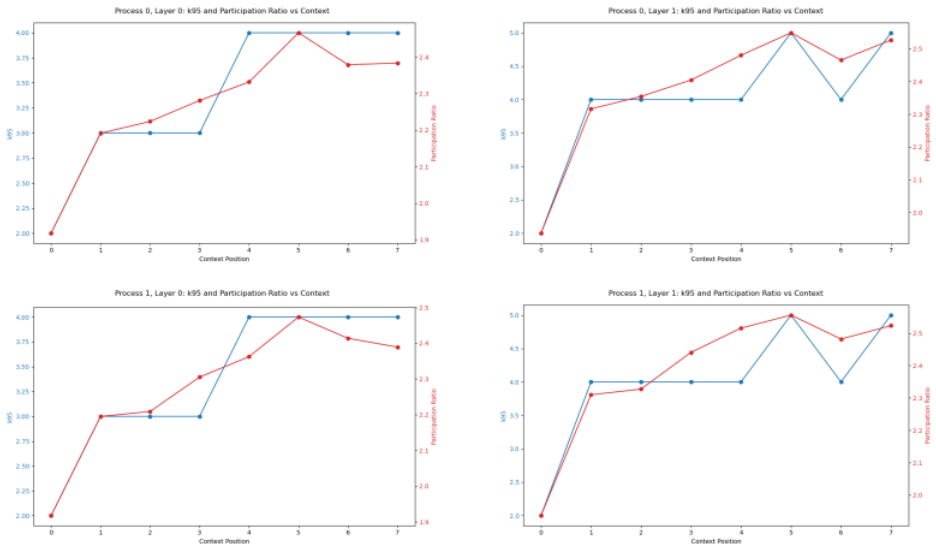

Figure 15: **Dimensionality vs Context Position for Long Context**: Plotted are k95 and PR as a function of context position.